



UNIVERSITAT POLITÈCNICA DE CATALUNYA
BARCELONATECH

Centre de la Imatge i la Tecnologia Multimèdia

Impact on programming

3.2 Surviving a harsh environment

Sergi Abadal (abadal@ac.upc.edu)

Xarxes i Jocs On-line – 2025-2026
Grau en Disseny i Desenvolupament de Videojocs

1. Introduction
2. The protocol stack
3. Impact on programming of on-line games (4 sessions)
 - Summary and general aspects
 - **Latency, jitter, and reliability**
 - Latency hiding techniques
 - Scalability
4. Other stories

Remember from last class

- On-line games basically need to transmit and receive data related (but not limited to) changes of state, whatever that means depending on the type of game, purpose, context:
 - Movements or attacks within the game
 - Control messages to start or finish games
 - Keep-alive updates
 - Sanity checks
 - Update of leaderboards or other game services
 - Micro-transactions
 - Stream of the actual video (e.g. Google Stadia)

Remember from last class

- Requirements depend on the type of message
 - Micro-transactions need to be reliable and secure (encrypted) as they involve payment data.
 - Messages for game services generally need to be reliable, and they are not latency sensitive.
 - Movements in games have different requirements depending on the type of game
 - **Turn-based:** little latency sensitivity (seconds)
 - **Real-Time Strategy (RTS):** some latency sensitivity (up to 1 second)
 - **Massive Multiplayer Games:** some latency sensitivity, also scalability
 - **Shooters or sport games:** a lot of latency sensitivity (up to 150 ms)
 - **AR/VR games:** super latency sensitive (<100 ms) to avoid cyber-sickness

Remember from last class



CITM

*How do we program the game's netcode
so that we can hide the detrimental
effects of the network and still maintain
as many functionalities as possible?*



UNIVERSITAT POLITÈCNICA DE CATALUNYA
BARCELONATECH

Centre de la Imatge i la Tecnologia Multimèdia

Preamble

The invariant of on-line games

USER
GAME
NETW

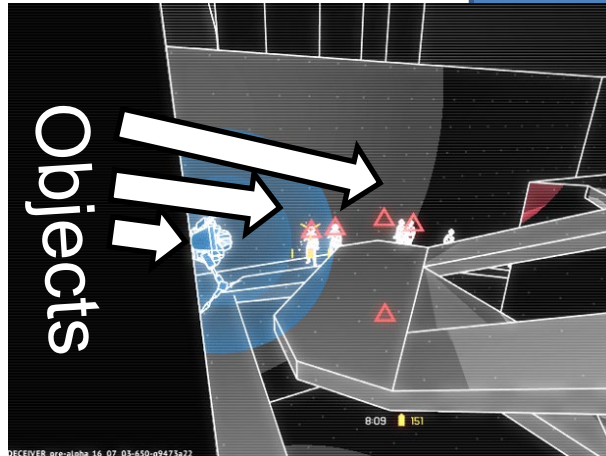


Player 1, click
Player 1 changes state



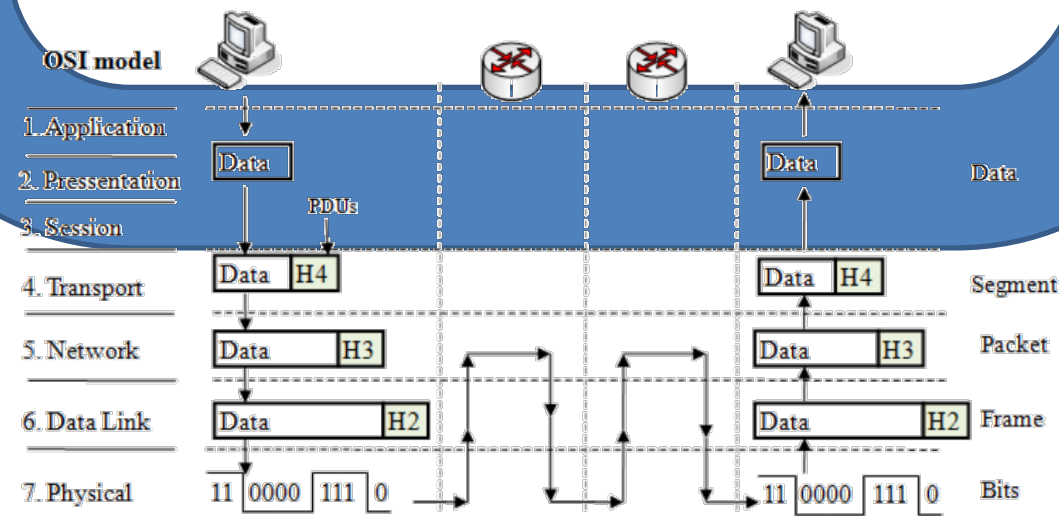
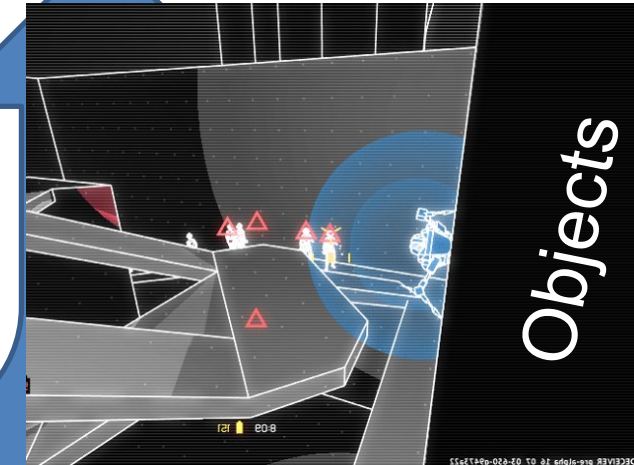
Player 2 receives info
Player 2 changes state

World (state)



We want the consequences of actions from player 1 to the state of the simulated world to be seen equally in all players.

World (state)



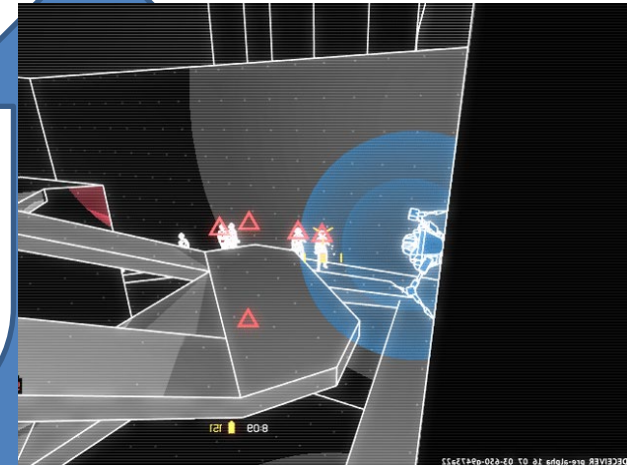
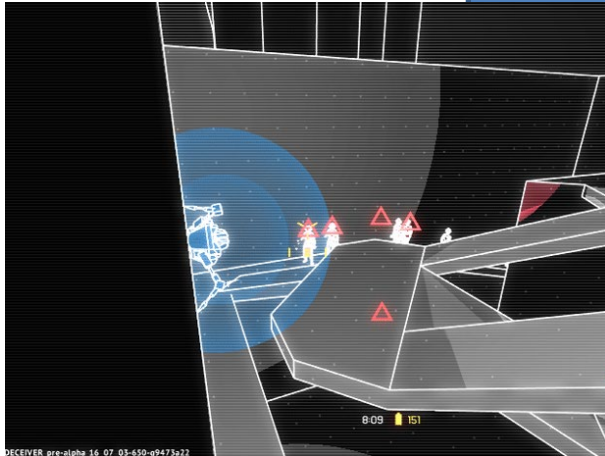
The invariant of on-line games



Player 1, click
Player 1 changes state



Player 2 receives info
Player 2 changes state



LATENCY, JITTER, LOSSES

Reasonable objectives? From HALO



CITM

Numbers from Reach

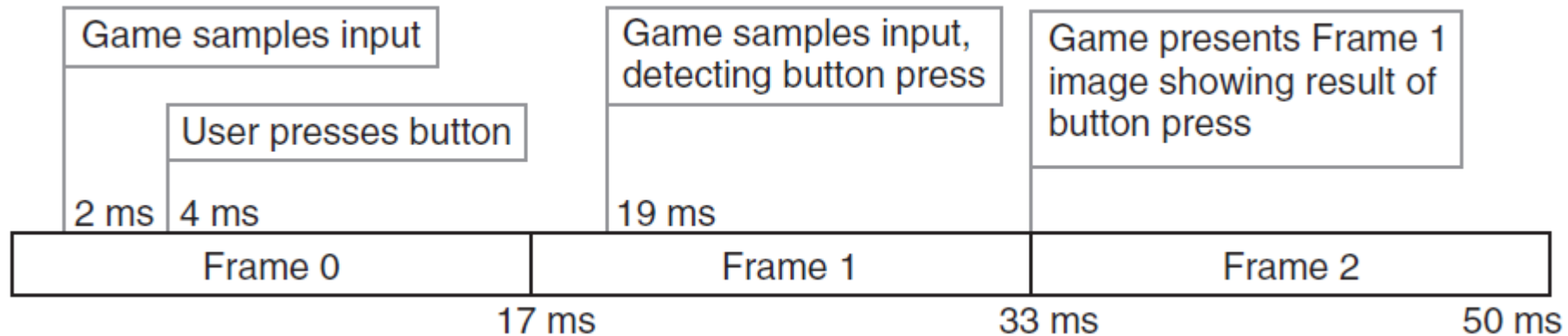
250kbits/s	Minimum total upstream for the host of a solid 16 player game
675kbits/s	Maximum total upstream bandwidth use from a single peer
45kbits/s	Maximum bandwidth sent to one client from a host
1kbit/s	Host upstream required to replicate one biped to one client at combat quality
10hz	Minimum packet rate for solid gameplay
100ms/200ms	Maximum latency for close-quarters gameplay for tournament/casual
133ms/300ms	Maximum latency for ranged gameplay for tournament/casual

- **Latency:** amount of time between an observable cause and its observable effect
 - From a mouse click to a unit responding in an RTS
 - From user moving its head and VR headset responding
- Unavoidable. Your objective is to minimize it.
- Sources
 - Non-network
 - Network

Non-network latency

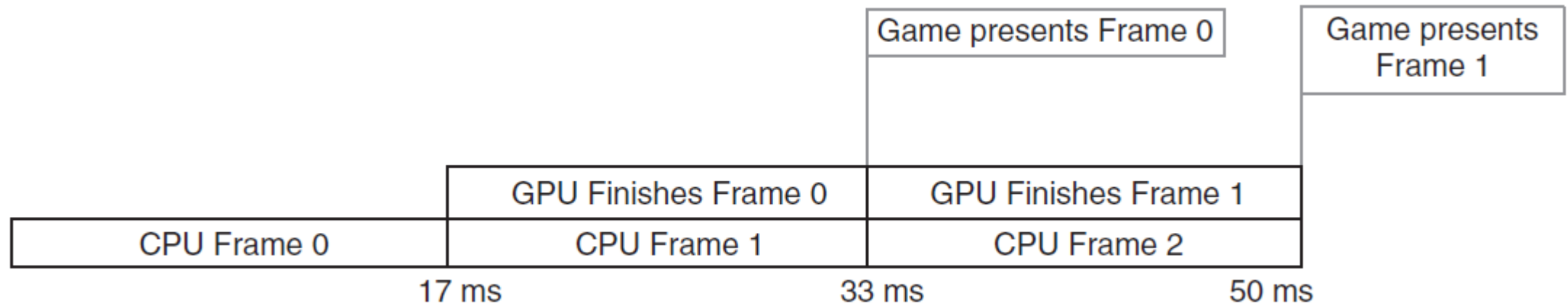


- **Input sampling latency:** time between your click and the game noticing. Half a frame duration ~ 8 ms



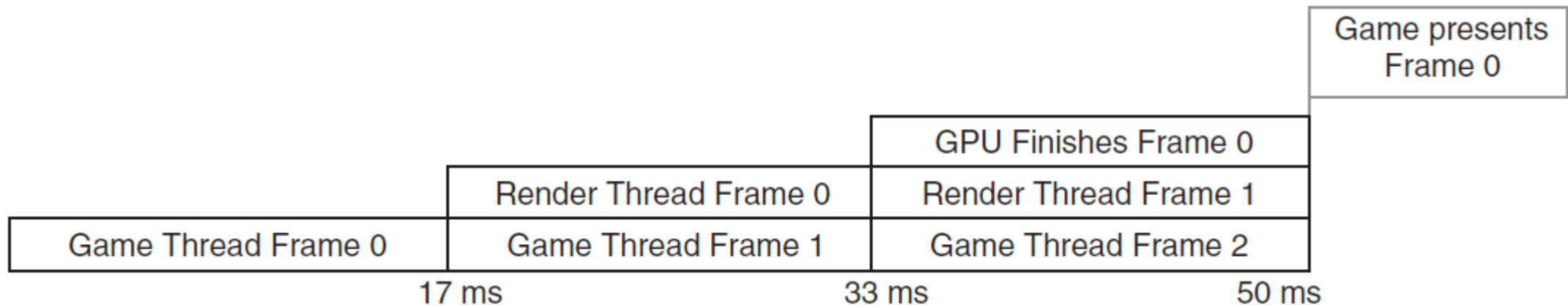
Non-network latency

- **Input sampling latency:** time between your click and the game noticing. Half a frame duration ~ 8 ms
- **Render pipeline latency:** time between CPU issuing commands to GPU and GPU reacting. One frame ~ 16 ms



Non-network latency

- **Input sampling latency:** time between your click and the game noticing. Half a frame duration ~ 8 ms
- **Render pipeline latency:** time between CPU issuing commands to GPU and GPU reacting. One frame ~ 16 ms
- **Multithreaded render pipeline latency:** separate render threads to update the world adds up. One frame ~ 16 ms



- **Input sampling latency:** time between your click and the game noticing. Half a frame duration ~ 8 ms
- **Render pipeline latency:** time between CPU issuing commands to GPU and GPU reacting. One frame ~ 16 ms
- **Multithreaded render pipeline latency:** separate render threads to update the world adds up. One frame ~ 16 ms
- **VSync:** synchronization between the render and the display framerate, needs to match. One frame ~ 16 ms

- **Input sampling latency:** time between your click and the game noticing. Half a frame duration ~ 8 ms
- **Render pipeline latency:** time between CPU issuing commands to GPU and GPU reacting. One frame ~ 16 ms
- **Multithreaded render pipeline latency:** separate render threads to update the world adds up. One frame ~ 16 ms
- **VSync:** synchronization between the render and the display framerate, needs to match. One frame ~ 16 ms
- **Display lag and pixel response time.**

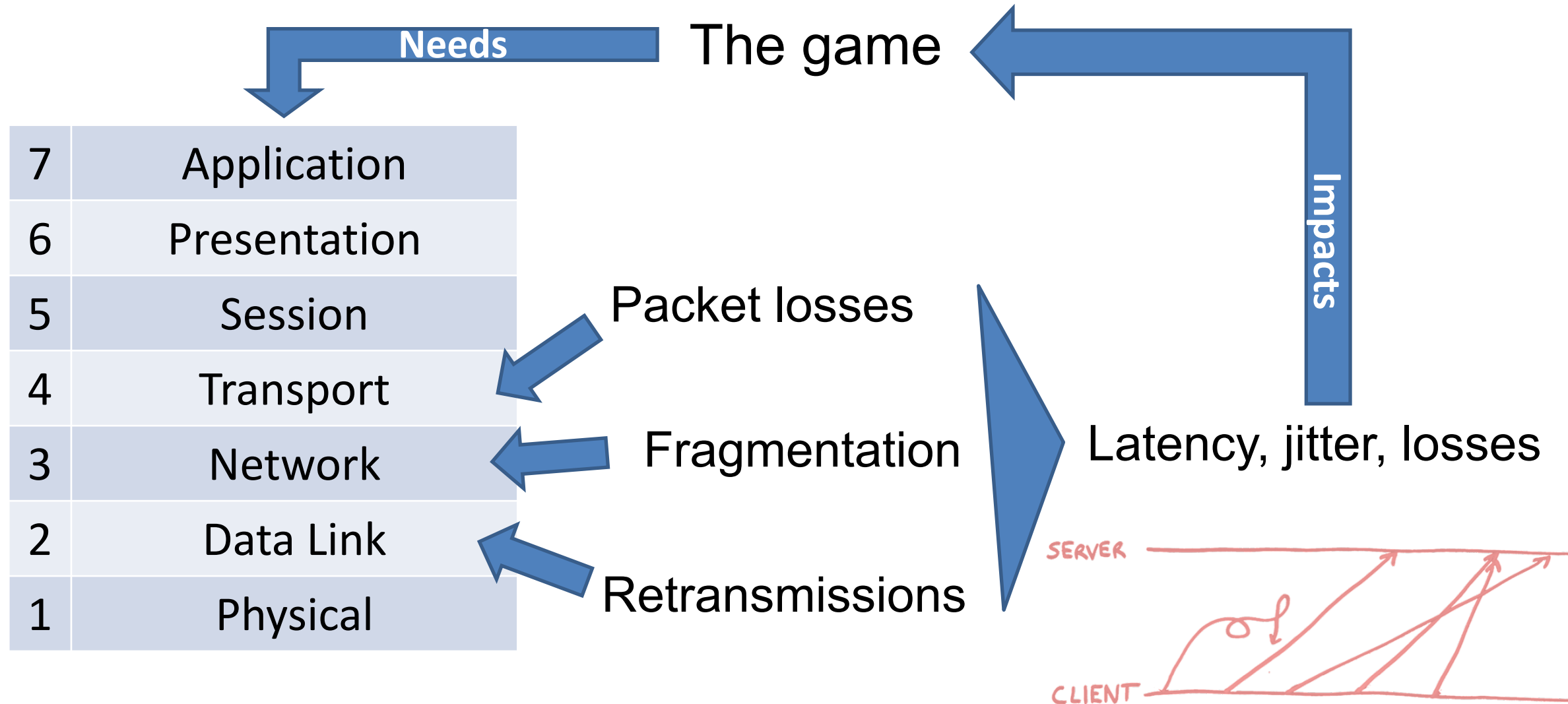


UNIVERSITAT POLITÈCNICA DE CATALUNYA
BARCELONATECH

Centre de la Imatge i la Tecnologia Multimèdia

Problems caused by the network

What causes problems?



- **Physical and link layers:** low speed in areas without 5G or good WiFi coverage.
- **Network layer:** NAT problems, IP packets losing order because they are treated independently, fragmentation.
- **Transport layer:** UDP is fast but unreliable and does not have congestion control (key to minimize packet losses)
- **Session:** implementing security through the adequate protocols (TLS) has an overhead.
- **Application:** small packets with high header overhead or inefficient object replication consuming high bandwidth



UNIVERSITAT POLITÈCNICA DE CATALUNYA
BARCELONATECH

Centre de la Imatge i la Tecnologia Multimèdia

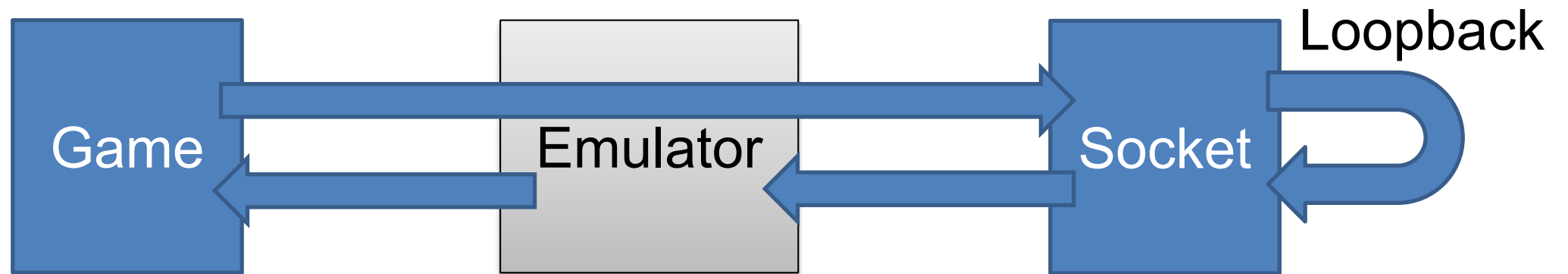
How to simulate the network and its problems?

Emulating real-world conditions



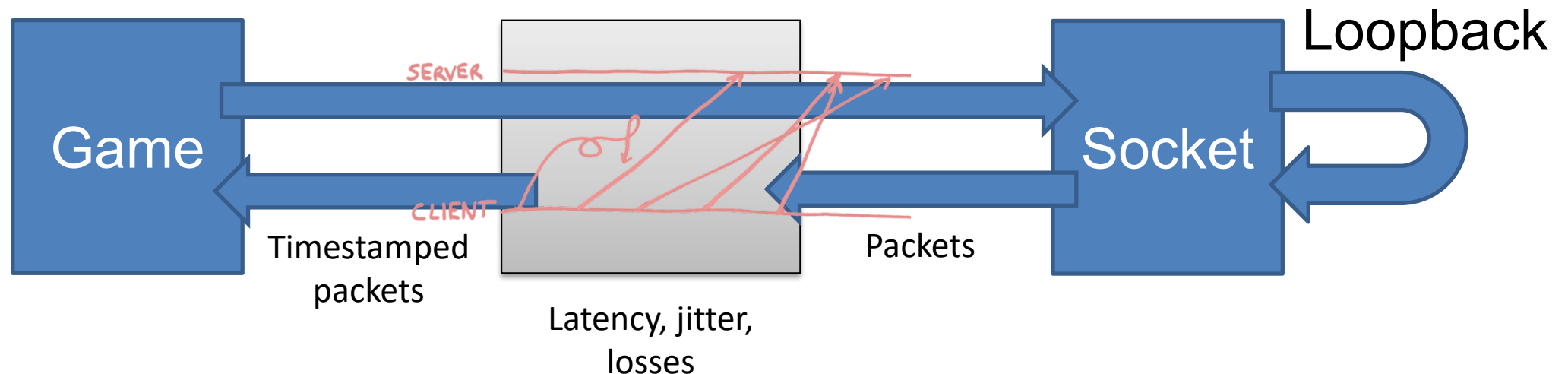
CITM

- It is important to be capable of testing games without having to constantly deploying them into a network.
- This can be done with a module between the game and the socket.



Emulating real-world conditions

- The emulator takes incoming packets and
 - Loses some of them
 - Delays and jitters those who are not lost
- Timestamped packets are put into a queue that the game will read in the appropriate times



Emulating real-world conditions

Listing 7.14 Simulating Loss, Latency, and Jitter

```
void RLSimulator::ReadIncomingPacketsIntoQueue()
{
    char packetMem[1500];
    int packetSize = sizeof(packetMem);
    InputMemoryBitStream inputStream(packetMem, packetSize * 8);
    SocketAddress fromAddress;

    while(receivedPackedCount < kMaxPacketsPerFrameCount)
    {
        int cnt = mSocket->ReceiveFrom(packetMem, packetSize, fromAddress);
        if(cnt == 0)
        {
            break;
        }
        else if(cnt < 0)
        {
            //handle error
        }
        else
        {
```



This is just the
regular code
to read from
the socket

Emulating real-world conditions



```
else  
{
```

Random number to drop a packet

```
//now, should we process the packet?
```

```
if(RoboMath::GetRandomFloat() >= mDropPacketChance)
```

Parameter: probability of dropping a packet

```
{
```

```
//we made it, queue packet for later processing
```

```
float simulatedReceivedTime =
```

```
    Timing::sInstance.GetTimef() +
```

```
    mSimulatedLatency + Parameter: average latency
```

```
    (RoboMath::GetRandomFloat() - 0.5f) * Random number to apply jitter (can be negative)
```

```
    mDoubleSimulatedMaxJitter; Parameter: jitter level
```

```
//keep list sorted by simulated receive time
```

```
auto it = mPacketList.end();
```

```
while(it != mPacketList.begin())
```

```
{
```

```
    --it;
```

```
    if(it->GetReceivedTime() < simulatedReceivedTime)
```

```
    {
```

```
        //time comes after this element, so inc and break
```

```
        ++it;
```

```
        break;
```

```
    }
```

```
}
```

```
mPacketList.emplace(it, simulatedReceivedTime,
```

```
                    inputStream, fromAddress);
```

```
    }  
}
```

Here is where we lose and timestamp the packets. We keep the queue ordered by timestamp

Emulating real-world conditions

```
void RLSimulator::ProcessQueuedPackets()  
{  
    float currentTime = Timing::sInstance.GetTimef();  
    //look at the front packet...  
    while(!mPacketList.empty())  
    {  
        ReceivedPacket& packet = mPacketList.front();  
        //is it time to process this packet?  
        if(currentTime > packet.GetReceivedTime())  
        {  
            ProcessPacket(packet.GetInputStream(),  
                           packet.GetFromAddress());  
            mPacketList.pop_front();  
        }  
        else  
        {  
            break;  
        }  
    }  
}
```

The game can only read the packets
if the time is right

This is a function
to read the packets
from the time-
stamped queue



UNIVERSITAT POLITÈCNICA DE CATALUNYA
BARCELONATECH

Centre de la Imatge i la Tecnologia Multimèdia

Implementing custom reliability

Lab Session 8

Reliability over the UDP Protocol

The reliability problem



CITM

- The network introduces delays, jitter, and losses.
- By default, games will not tolerate this
 - Inconsistent state
 - Lag, bad quality of experience
 - Glitches

<https://youtu.be/wlKEwF2eX6Q> - OK

<https://youtu.be/Rngf78IBROA> - KO

- Who can take care of end-to-end reliability?
 - ~~Physical Layer~~
 - ~~Link Layer~~
 - ~~Network Layer~~
 - Transport Layer ✓
 - Upper Layers ✓

The reliability solution: Transport Layer



CITM

Column Heading	TCP	UDP
Reliability	Native. Everything is delivered and processed in the order it was sent.	None. Requires custom implementation but allows fine-grained reliability.
Flow control	Will automatically slow down rate of transmission if packets are getting dropped.	None. Requires custom flow and congestion control if desired.
Memory requirements	OS must keep copies of all data sent until it is acknowledged.	Custom implementation must decide what data to keep around and what to discard immediately. Memory managed at application level.
Router prioritization	May be prioritized over UDP packets.	May be dropped before TCP packets.

Robust, Stable, Tested

Fast, “blank canvas”

Strict, not manageable

Not reliable

TCP or UDP?



- **Only choose TCP in turn-based games**, where you want all your data and you want it ordered. Why?
- TCP is reliable, stable, robust, but you cannot control the degree of reliability, ordering. When is this a problem?
 - With data and different levels of priority (headshot vs sound)
 - With different streams of important data (headshot vs chat)
 - With data that becomes stale (position, health points)
- Hard to manage, troubleshoot, because TCP allocations are made by the OS (not the application).

TCP or UDP?



- **Only choose TCP in turn-based games**, where you want all your data and you want it ordered. Why?
- Because the consequence of all the previous problems leads to glitchy, laggy system if the delay and error rate start going up with bad quality of experience
 - <https://youtu.be/fiMEQpN2rZQ> - TCP, 100 ms, 1%
 - <https://youtu.be/Rngf78IBROA> - TCP, 250 ms, 5%

TCP or UDP?



CITM

- **Choose UDP in the majority of cases**, where you need varying levels of reliability and you need it NOW.
- UDP is not reliable, but gives the designer freedom to do TCP things at the application layer and control on how to:
 - Establish connections
 - Group data into packets
 - Implement priority
 - Proactively send multiple copies of important packets
 - Resend only the packets that need it
 - Order only packets that need it

- First step on reliability is to know the connection status of other players at all times
- This is done via **Virtual UDP Connections**

Virtual UDP Connection

UDP is not TCP:

- Not connection oriented
- No knowledge of when the client is "alive"

How to mitigate?

- Ping packets and timeouts

From the lab sessions

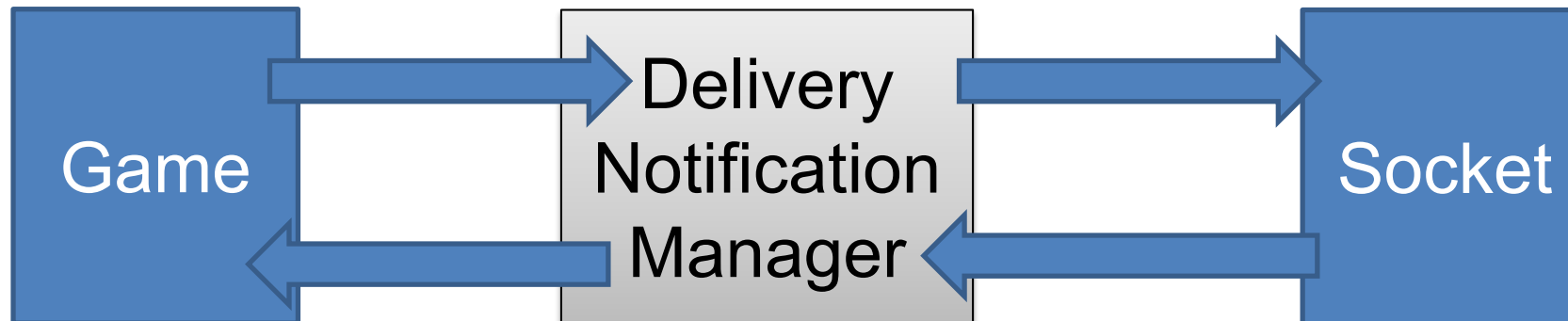
Packet Delivery Notification



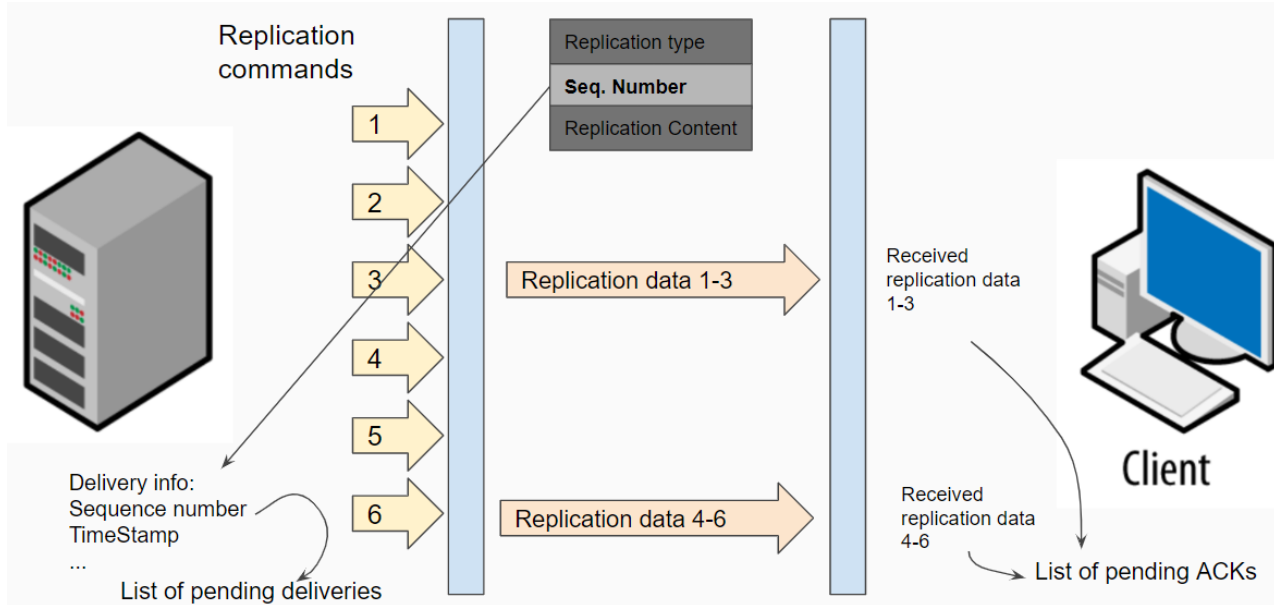
CITM

- Next steps on reliability are to
 - Know if packets have reached the destination
 - Provide a way to order packets, if needed
- This is given by a **Delivery Notification Manager (DNM)**

Flexible! It's the designer's decision to retransmit or order incoming packets based on the DNM info

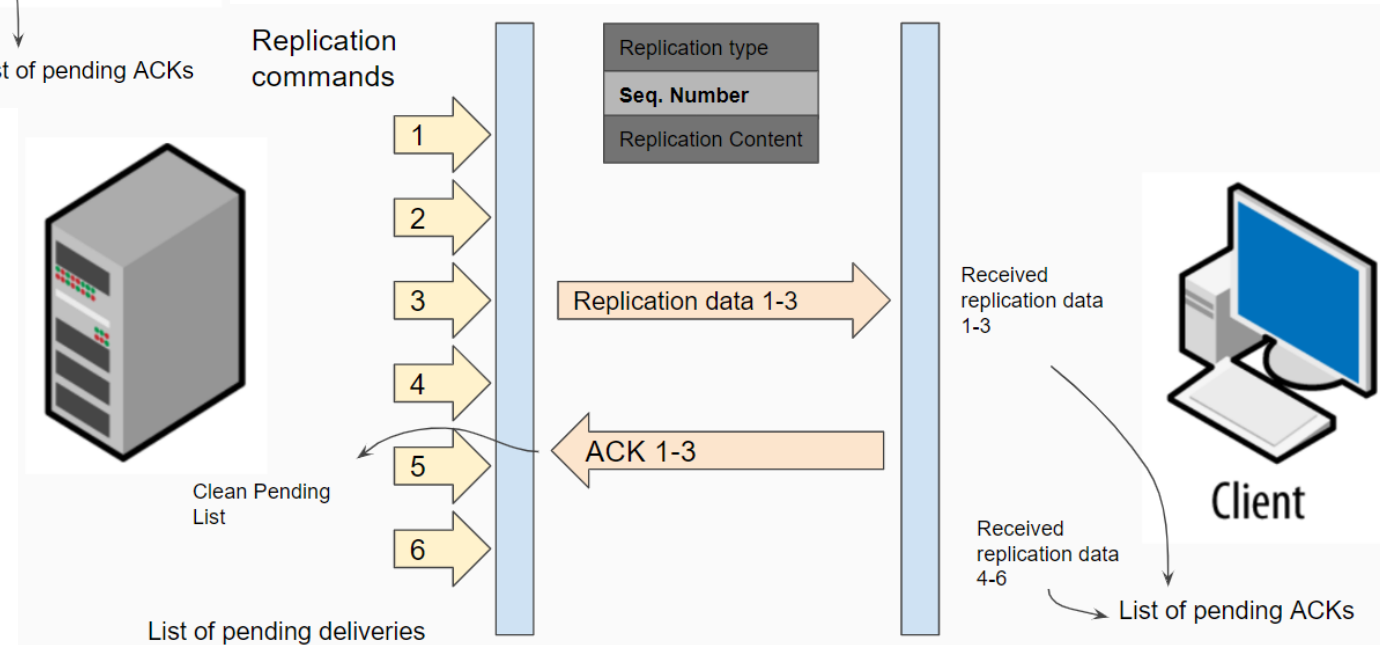


Delivery Notification System



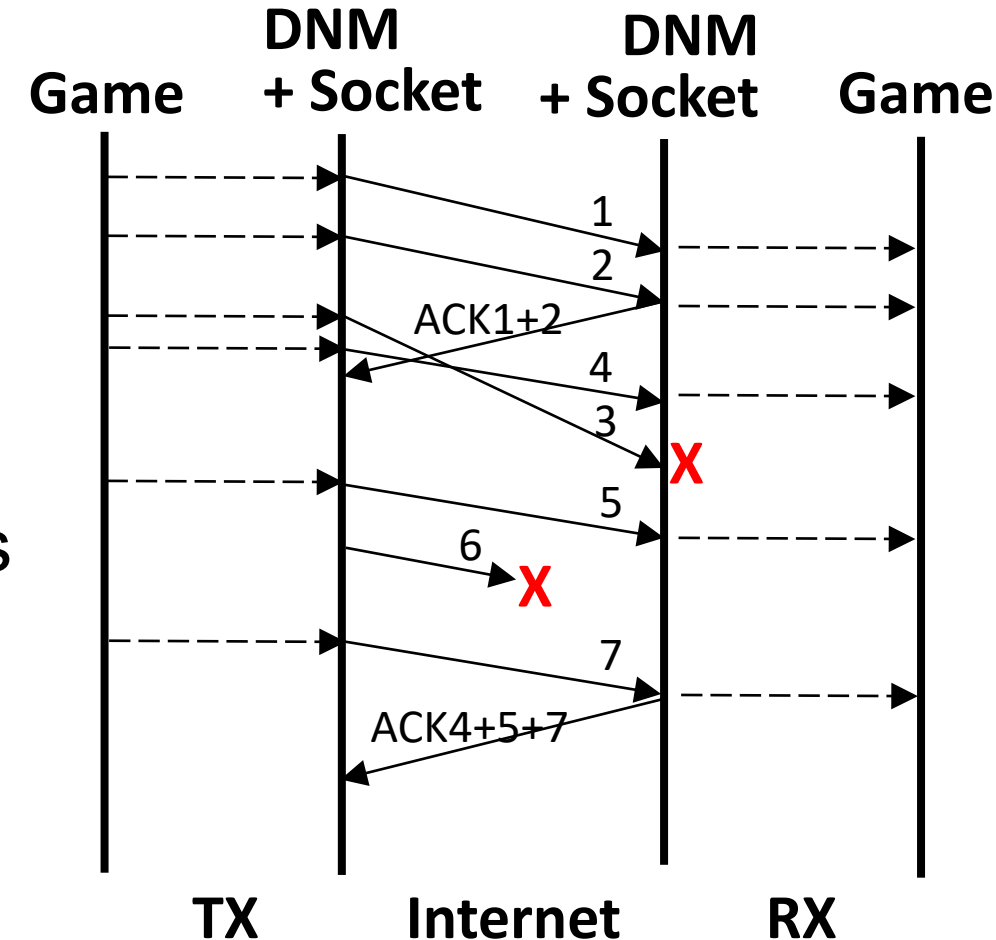
- Also called “Delivery Notification System” in the lab sessions

From the lab sessions



Packet Delivery Notification

- The DNM does three things:
 1. TX: Uniquely identify and tag outgoing packets so that delivery status can be associated with the correct message
 2. RX: Examine incoming packets and send acknowledgments for the packets that it decides to process
 3. TX: Process acknowledgments and notify the game on what has been delivered and what was dropped



Tagging Outgoing Packets



CITM

Copy of the packet that is held
in the DNM until acknowledged

Original packet to be
sequenced and sent

Listing 7.1 Tagging a Packet with a Sequence Number

```
InFlightPacket* DeliveryNotificationManager::WriteSequenceNumber(  
    OutputMemoryBitStream& inPacket)  
{  
    PacketSequenceNumber sequenceNumber = mNextOutgoingSequenceNumber++;  
    inPacket.Write(sequenceNumber);  
  
    ++mDispatchedPacketCount;  
  
    mInFlightPackets.emplace_back(sequenceNumber);  
    return &mInFlightPackets.back();  
}
```

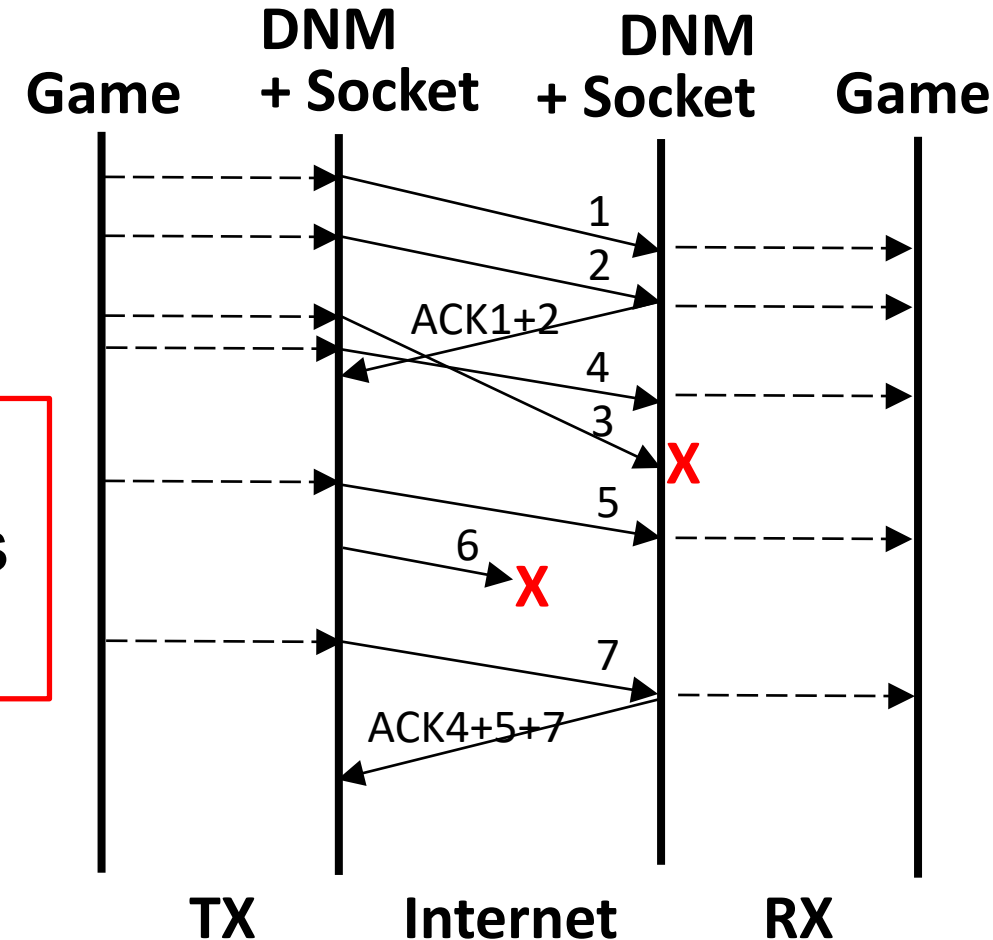
Unique
sequence
number

List of packets
copied and held

Typedef it: decide if small or large
(depends on number of generated packets)

Packet Delivery Notification

- The DNM does three things:
 1. TX: Uniquely identify and tag outgoing packets so that delivery status can be associated with the correct message
 2. RX: Examine incoming packets and send acknowledgments for the packets that it decides to process
 3. TX: Process acknowledgments and notify the game on what has been delivered and what was dropped

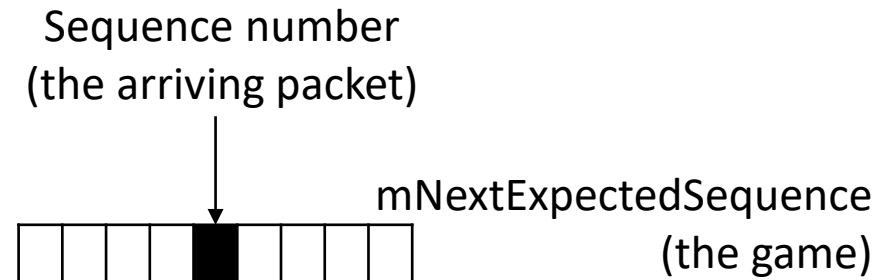


Receiving packets...

Retrieve sequence number
from incoming packet

Listing 7.2 Processing an Incoming Sequence Number

```
bool DeliveryNotificationManager::ProcessSequenceNumber(  
    InputMemoryBitStream& inPacket)  
{  
    PacketSequenceNumber sequenceNumber;  
  
    inPacket.Read(sequenceNumber);  
    if(sequenceNumber == mNextExpectedSequenceNumber)  
    {  
        //is this expected? add ack to the pending list and process packet  
        mNextExpectedSequenceNumber = sequenceNumber + 1;  
        AddPendingAck(sequenceNumber);  
    }  
}
```



OK. Expected packet. Mark it as pending to process, jump to next sequence number.



Receiving packets...

Retrieve sequence number
from incoming packet

Listing 7.2 Processing an Incoming Sequence Number

```
bool DeliveryNotificationManager::ProcessSequenceNumber(  
    InputMemoryBitStream& inPacket)  
{  
    PacketSequenceNumber sequenceNumber;  
  
    inPacket.Read(sequenceNumber);  
    if(sequenceNumber == mNextExpectedSequenceNumber)  
    {  
        //is this expected? add ack to the pending list and process packet  
        mNextExpectedSequenceNumber = sequenceNumber + 1;  
        AddPendingAck(sequenceNumber);  
    }  
}
```

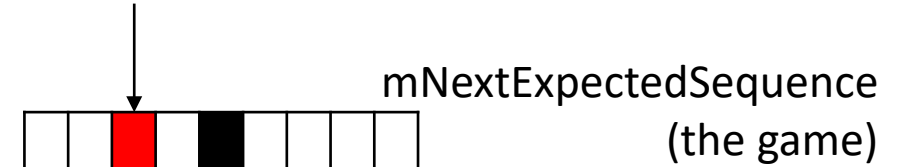


OK. Expected packet. Mark it as pending to process, jump to next sequence number.



Receiving packets...

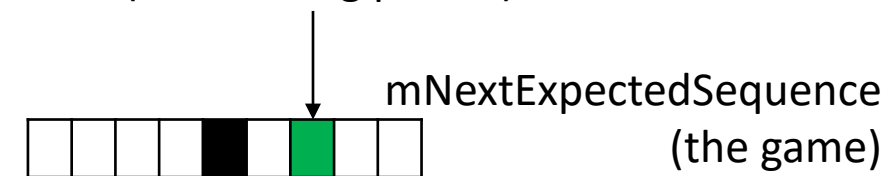
Sequence number
(the arriving packet)



KO. Old packet (out of order delivery). Do nothing.

OK. Valid packet after some dropped ones. Sequence number advances by several units.

Sequence number
(the arriving packet)



```
return true;
}
//is sequence number < current expected? Then silently drop old packet.
else if(sequenceNumber < mNextExpectedSequenceNumber)
{
    return false;
}
//otherwise, we missed some packets
else if(sequenceNumber > mNextExpectedSequenceNumber)
{
    //consider all skipped packets as dropped, so
    //our next expected packet comes after this one...
    mNextExpectedSequenceNumber = sequenceNumber + 1;
    //add an ack for the packet and process it
    //when the sender detects break it acks, it can resend
    AddPendingAck(sequenceNumber);
    return true;
}
}
```

Receiving packets...



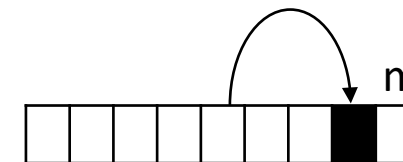
```
    return true;
}
//is sequence number < current expected? Then silently drop old packet.
else if(sequenceNumber < mNextExpectedSequenceNumber)
{
    return false;
}
//otherwise, we missed some packets
else if(sequenceNumber > mNextExpectedSequenceNumber)
{
    //consider all skipped packets as dropped, so
    //our next expected packet comes after this one...
    mNextExpectedSequenceNumber = sequenceNumber + 1;
    //add an ack for the packet and process it
    //when the sender detects break it acks, it can resend
    AddPendingAck(sequenceNumber);
    return true;
}
}
```

KO. Old packet (out of order delivery). Do nothing.



mNextExpectedSequence
(the game)

OK. Valid packet after some dropped ones. Sequence number advances by several units.



mNextExpectedSequence
(the game)

...and sending ACKs

Listing 7.2 Processing an Incoming Sequence Number

```
bool DeliveryNotificationManager::ProcessSequenceNumber(
    InputMemoryBitStream& inPacket)
{
    PacketSequenceNumber    sequenceNumber;

    inPacket.Read(sequenceNumber);
    if(sequenceNumber == mNextExpectedSequenceNumber)
    {
        //is this expected? add ack to the pending list and process packet
        mNextExpectedSequenceNumber = sequenceNumber + 1;
        AddPendingAck(sequenceNumber);
```

Instead of sending 1 ACK per message, we buffer them for later

```
void DeliveryNotificationManager::AddPendingAck(
    PacketSequenceNumber inSequenceNumber)
{
    if(mPendingAcks.size() == 0 ||
        !mPendingAcks.back().ExtendIfShould(inSequenceNumber))
    {
        mPendingAcks.emplace_back(inSequenceNumber);
    }
}
```

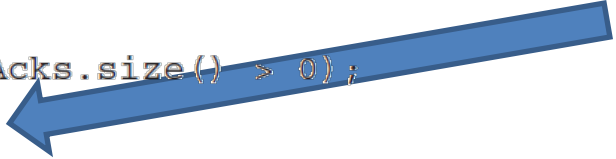
...and sending ACKs

When the receiver is ready to reply,
it sends the accumulated ACKs by calling this function

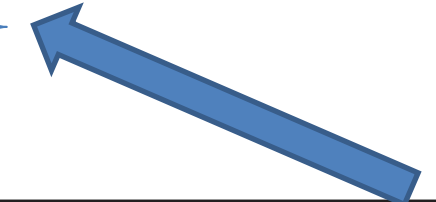
Listing 7.5 Writing Pending Acknowledgments

```
void DeliveryNotificationManager::WritePendingAcks(  
    OutputMemoryBitStream& inPacket)  
{  
    bool hasAcks = (mPendingAcks.size() > 0);  
    inPacket.Write(hasAcks);  
    if (hasAcks)  
    {  
        mPendingAcks.front().Write(inPacket);  
        mPendingAcks.pop_front();  
    }  
}
```

First bit of the packet
indicates if it has ACKs
(kind of header)

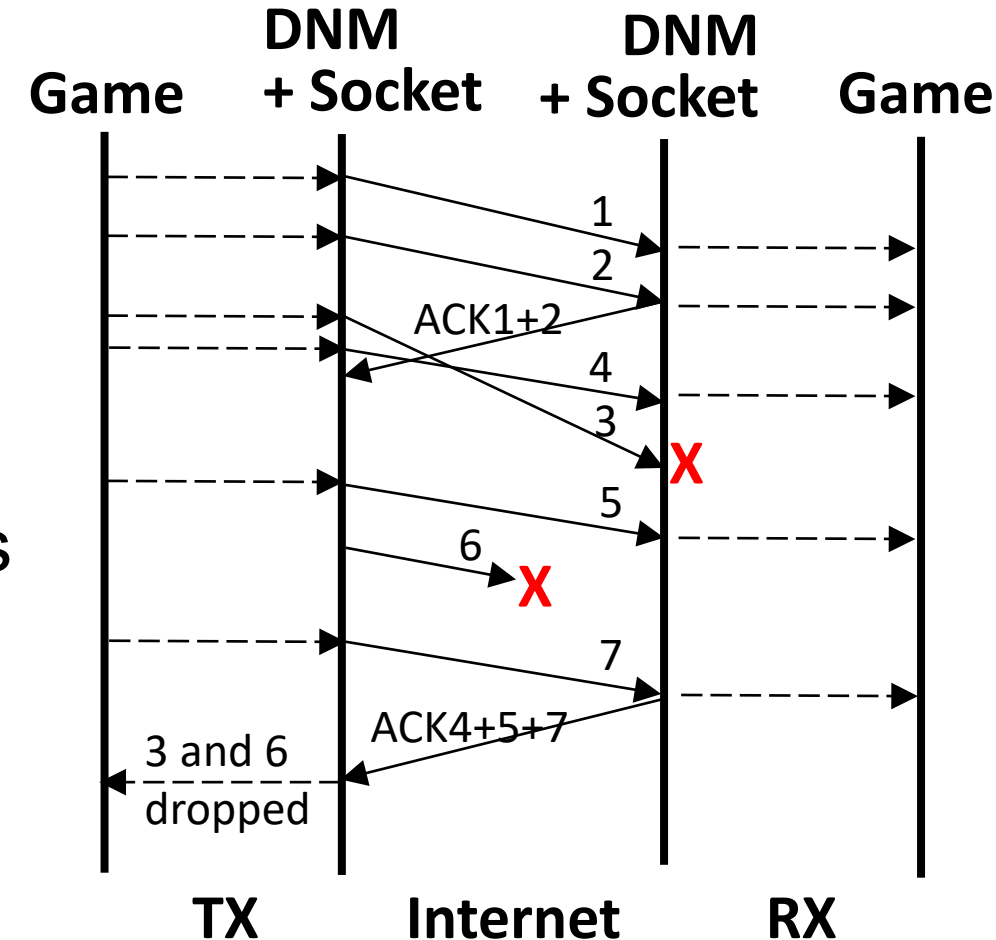


The method appends as
many ACKs as the contained
in a single mPendingAcks
(decided by designer)



Packet Delivery Notification

- The DNM does three things:
 1. TX: Uniquely identify and tag outgoing packets so that delivery status can be associated with the correct message
 2. RX: Examine incoming packets and send acknowledgments for the packets that it decides to process
 3. TX: Process acknowledgments and notify the game on what has been delivered and what was dropped



Receiving acknowledgments...

Listing 7.6 Processing the Acknowledgments

```
void DeliveryNotificationManager::ProcessAcks(  
    InputMemoryBitStream& inPacket)  
{  
    bool hasAcks;  
    inPacket.Read(hasAcks);  
    if (hasAcks)  
    {  
        AckRange ackRange;  
        ackRange.Read(inPacket);  
        //for each InFlightPacket with seq# <= nextAckdSeqNum, handle failure...  
        PacketSequenceNumber nextAckdSequenceNumber =  
            ackRange.GetStart();  
        uint32_t onePastAckdSequenceNumber =  
            nextAckdSequenceNumber + ackRange.GetCount();  
        while (nextAckdSequenceNumber < onePastAckdSequenceNumber &&  
            !mInFlightPackets.empty())
```

Distinguish packets containing ACKs
from those containing data

We read the ACKs and
start with the first one

Process all the ACKs
contained in the packet



Receiving acknowledgments...



```
while(nextAckdSequenceNumber < onePastAckdSequenceNumber &&
      !mInFlightPackets.empty())
{
    const auto& nextInFlightPacket = mInFlightPackets.front();
    //if the packet seq# < ack seq#, we didn't get an ack for it,
    //so it probably wasn't delivered
    PacketSequenceNumber nextInFlightPacketSequenceNumber =
        nextInFlightPacket.GetSequenceNumber();
    if(nextInFlightPacketSequenceNumber < nextAckdSequenceNumber)
    {
        //copy this so we can remove it before handling-
        //dependent modules shouldn't find it if seeing what's live
        auto copyOfInFlightPacket = nextInFlightPacket;
        mInFlightPackets.pop_front();
        HandlePacketDeliveryFailure(copyOfInFlightPacket);
    }
    else if(nextInFlightPacketSequenceNumber==
            nextAckdSequenceNumber)
    {
        HandlePacketDeliverySuccess(nextInFlightPacket);
        //received!
        mInFlightPackets.pop_front();
        ++nextAckdSequenceNumber;
    }
    else if(nextInFlightPacketSequenceNumber>
            nextAckdSequenceNumber)
    {
        //somehow part of this range was already removed
        //(maybe from timeout) check rest of range
        nextAckdSequenceNumber = nextInFlightPacketSequenceNumber;
    }
}
```

For each ACK, we obtain the ACK number and compare with the SEQ of the InFlightPackets (copies of yet unacknowledged sent packets)



KO. The inFlight packet has not been ACKed (probably it did not arrive). Remove from list but handle the error.



OK. Remove the inFlight packet, and make the game process the packet.



OK. Some of the inFlight packets were already removed and still they were ACKed (delayed). We do nothing.

...and delivering status

Listing 7.8 Tracking Status

```
void DeliveryNotificationManager::HandlePacketDeliveryFailure(  
    const InFlightPacket& inFlightPacket)  
{  
    ++mDroppedPacketCount;  
    inFlightPacket.HandleDeliveryFailure(this);  
}  
  
void DeliveryNotificationManager::HandlePacketDeliverySuccess(  
    const InFlightPacket& inFlightPacket)  
{  
    ++mDeliveredPacketCount;  
    inFlightPacket.HandleDeliverySuccess(this);  
}
```

Keep statistics of the dropped and delivered packets to react in the future

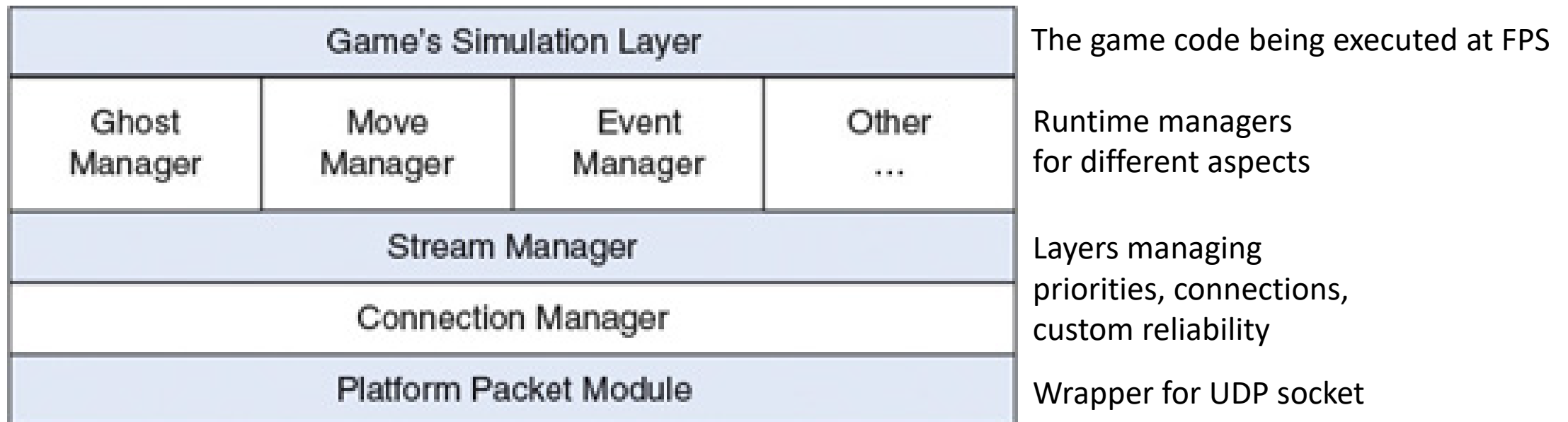
Let the game know that the packets were dropped or delivered so that it can react to those



The game decides whether to send the packet again, not do anything, create/update/destroy the replicated objects sent with that packet, etc...

Remember: Starsiege Tribes

- When ACKs are received or packets are dropped, the relevant ghost/move/event manager will be notified.
- They will individually decide what to do in that situation depending on how important is the data.



Videos (deterministic lockstep)



CITM

- <https://youtu.be/Rngf78IBROA> - TCP, 250 ms, 5%
- https://youtu.be/eO8_9SVdRis - UDP, 2 sec, 25%



UNIVERSITAT POLITÈCNICA DE CATALUNYA
BARCELONATECH

Centre de la Imatge i la Tecnologia Multimèdia

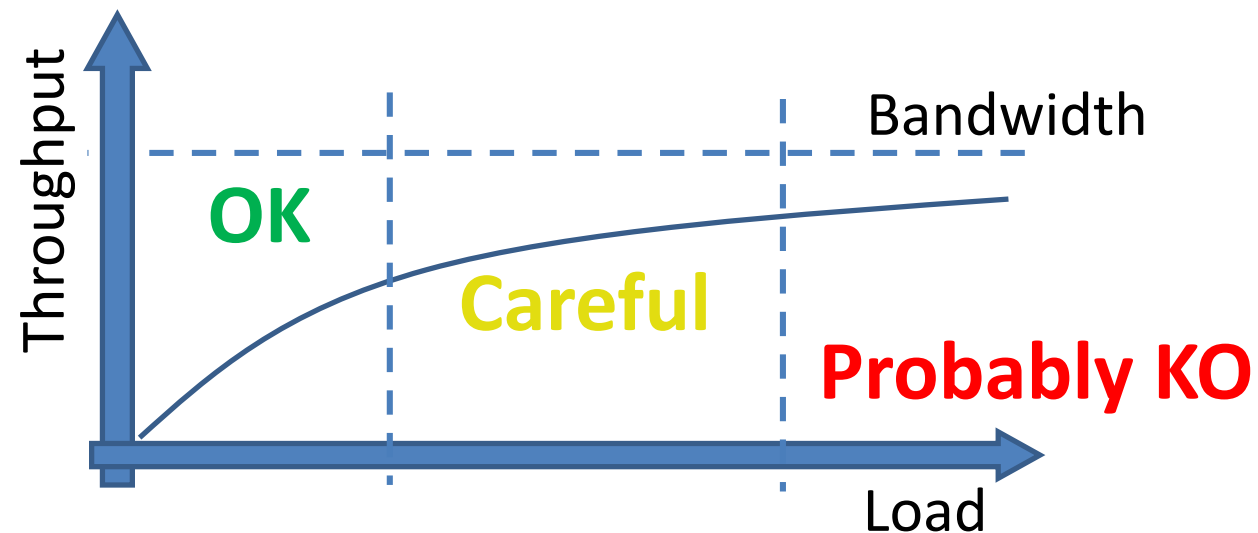
Use your connection wisely: Snapshot Compression

Bandwidth and Throughput



CITM

- **Load:** speed at which you want to send
- **Bandwidth:** theoretical maximum speed
- **Throughput:** actual speed that you are able to send



By sending data judiciously, we can save lots of time (especially important before having 100Mbps at home)

- **Sparse Array Compression:** you do not use a full fixed-size array (`char[128] = "Sergi"`)? Remove empty parts
- **Entropy Encoding:** are some values typical and can be taken by default? Use that to transmit less.
- **Fixed Point:** do you know range and resolution requirement? Use it to turn into integers.
- **Geometry Compression:** do you use quaternions? Maybe you can just send three numbers

TCP



UDP

<https://wiki.vg/Protocol>

Cortesía de:
Biel Rabasa
Alejandro Giralt
Roger Salas
Rubén Ayora
(XJO 2023-2024)

Sparse Array Compression



CITM

```
void RoboCat::Write(OutputMemoryStream& inStream) const
{
    ...//serialize other fields up here

    uint8_t nameLength =
        static_cast<uint8_t>(strlen(mName));
    inStream.Write(nameLength);
    inStream.Write(mName, nameLength);
    ...
}
```

Length + the useful part of the string
rather than the entire string.

Exercise – Sparse Array Compression



CITM

We store the name of our character in a `char[128]` array. We apply sparse array compression. Which is the relative size of the compressed data if the character name has, in average 6.4 characters?

(Use `uint8_t` for the name length)



Exercise – Sparse Array Compression



CITM

We store the name of our character in a `char[128]` array. We apply sparse array compression. Which is the relative size of the compressed data if the character name has, in average 6.4 characters?

$$\frac{6.4 * \text{sizeof}(\text{char}) + \text{sizeof}(\text{uint8})}{128 * \text{sizeof}(\text{char})} = \frac{6.4 * 8 + 8}{128 * 8} = \mathbf{5.7\%}$$

Sparse array compression

Reduce array sizes

Player Info Remove

Used by the server to remove players from the player list.

Packet ID	State	Bound To	Field Name		Field Type	Notes
0x3B	Play	Client	Number of Players		VarInt	Number of elements in the following array.
			Player	Player Id	Array of UUID	UUIDs of players to remove.

Int	4 bytes	± 2'147'483'647
UUID	16 bytes	UUID String

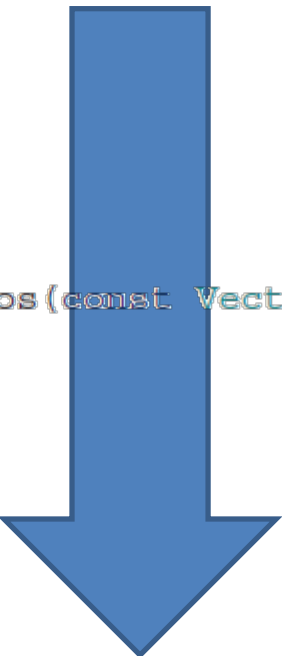


Entropy Encoding (I)

```
void OutputMemoryBitStream::Write(const Vector3& inVector)
{
    Write(inVector.mX);
    Write(inVector.mY);
    Write(inVector.mZ);
}

void OutputMemoryBitStream::WritePos(const Vector3& inVector)
{
    Write(inVector.mX);
    Write(inVector.mZ);

    if(inVector.mY == 0)
    {
        Write(true);
    }
    else
    {
        Write(false);
        Write(inVector.mY);
    }
}
```

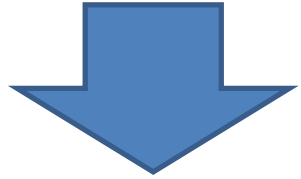


The more expected is a value (the lower the entropy), the more chances you will have to compress it.

In this example, we send the (X,Y,Z) position of our character. If it generally is on the ground, we assume Y=0 and send true/false plus the position if it is false.

Entropy Encoding (II)

```
void OutputMemoryBitStream::Write(const Vector3& inVector)
{
    Write(inVector.mX);
    Write(inVector.mY);
    Write(inVector.mZ);
}
```



```
if(inVector.mY == 0)
{
    Write(true);
    Write(true);
}
else if(inVector.mY == 100)
{
    Write(true);
    Write(false);
}
else
{
    Write(false);
    Write(inVector.mY);
}
}
```

The more expected is a value (the lower the entropy), the more chances you will have to compress it.

This can be generalized to a set of common positions. If character is generally on the ground and ceiling, then we can send two true/falsees plus the position if both are false.

Exercise – Entropy Encoding



CITM

Imagine that our character is 90% of the time on the floor. Which is the relative size of the sent data using entropy encoding on the $Y=0$ position? X , Y , and Z are 32-bit floating point variables.



Exercise – Entropy Encoding



CITM

Imagine that our character is 90% of the time on the floor. Which is the relative size of the sent data using entropy encoding on the Y=0 position? X, Y, and Z are 32-bit floating point variables.

$$\frac{2 * 32 + (0.1 * 33 + 0.9 * 1)}{3 * 32} = \frac{64 + 4.2}{96} = \mathbf{71\%}$$

Exercise – Entropy Encoding



CITM

Repeat the previous exercise now considering that the character is 90% of the time at the floor and 7% of the time in the ceiling, and we use both values to compress the position.



Exercise – Entropy Encoding



CITM

Repeat the previous exercise now considering that the character is 90% of the time at the floor and 7% of the time in the ceiling, and we use both values to compress the position.

$$\frac{2*32 + (0.9*2 + 0.07*2 + 0.03*33)}{3*32} = \frac{64 + 2.9}{96} = \mathbf{69.6\%}$$

Entropy encoding

Save common values

Double	8 bytes	Fractional numbers
Bool	1 bit	True or False

Set Player Position and Rotation

A combination of [Move Player Rotation](#) and [Move Player Position](#).

Packet ID	State	Bound To	Field Name	Field Type	Notes
0x1B	Play	Server	X	Double	Absolute position.
			Feet Y	Double	Absolute feet position, normally Head Y - 1.62.
			Z	Double	Absolute position.
			Yaw	Float	Absolute rotation on the X Axis, in degrees.
			Pitch	Float	Absolute rotation on the Y Axis, in degrees.
			On Ground	Boolean	True if the client is on the ground, false otherwise.



Entropy encoding

Save common values

Int	4 bytes	$\pm 2^{147}483'647$
Byte	1 byte	0 to 64
Bool	1 bit	True or False

Slot Data

The **Slot** data structure is how Minecraft represents an item and its associated data in the [Minecraft Protocol](#).

Slot



Name	Type	Meaning
Present	Boolean	True if there is an item in this position; false if it is empty.
Item ID	Optional VarInt	Omitted if present is false. The item ID . Item IDs are distinct from block IDs; see Data Generators for more information
Item Count	Optional Byte	Omitted if present is false.
NBT	Optional NBT	Omitted if present is false. If 0 (TAG_End), there is no NBT data, and no further data follows. Otherwise the byte is the start of an NBT blob as shown below:



Entropy encoding

Save common values



* 46 (17 empty, 29 full)

Int	4 bytes	$\pm 2^{147'483'647}$
Byte	1 byte	0 to 64
Bool	1 bit	True or False

Without compression:

$$46 * (\text{int}[32] + \text{byte}[8] + \text{byte}[8]) \\ = 2208 \text{ b}$$

With compression:

$$17 * (\text{bit}[1]) + 29 * (\text{bit}[1] + \text{int}[32] + \text{byte}[8] + \\ \text{byte}[8]) \\ = 1438 \text{ b}$$

Worst case with compression:

$$46 * (\text{bit}[1] + \text{int}[32] + \text{byte}[8] + \text{byte}[8]) \\ = 2254 \text{ b}$$

Fixed point

```
inline uint32_t ConvertToFixed(  
    float inNumber, float inMin, float inPrecision)  
{  
    return static_cast<uint32_t> (  
        (inNumber - inMin)/inPrecision);  
}  
  
inline float ConvertFromFixed(  
    uint32_t inNumber, float inMin, float inPrecision )  
{  
    return static_cast<float>(inNumber) *  
        inPrecision + inMin;  
}
```

Games generally execute floating point computations, although not all the variables need such precision.

If the limits and precision are known, we can create fixed point numbers that represent the value with less bits. Convert before sending and convert back when receiving.

Exercise – Fixed point



CITM

Assume that the X and Z components of our character take only values from -2000 to 2000 and with a precision of 0.1. How many bits can we use to represent them with fixed point compression?



Exercise – Fixed point



Assume that the X and Z components of our character take only values from -2000 to 2000 and with a precision of 0.1. How many bits can we use to represent them with fixed point compression?

$$N_{\text{val}} = \frac{2000 - (-2000)}{0.1} + 1 = 40001$$

$$N_{\text{bits}} = \log_2 N_{\text{val}} = 15.3$$

Exercise – Fixed point



CITM

Apply fixed point compression to the X and Z components in the example earlier. Which is the relative size of the compressed position with respect to the original three floating point units?



Exercise – Fixed point



CITM

Apply fixed point compression to the X and Z components in the example earlier. Which is the average relative size of the compressed position with respect to the original three floating point units?

$$\frac{2 * 16 + 2.93}{3 * 32} = 36\%$$

Fixed point

Limit range & resolution

Update Entity Position

This packet is sent by the server when an entity moves less then 8 blocks; if an entity moves more than 8 blocks [Teleport Entity](#) should be sent instead.

This packet allows at most 8 blocks movement in any direction, because short range is from -32768 to 32767. And $32768 / (128 * 32) = 8$.

Packet ID	State	Bound To	Field Name	Field Type	Notes
0x2C	Play	Client	Entity ID	VarInt	
			Delta X	Short	Change in X position as $(currentX * 32 - prevX * 32) * 128$
			Delta Y	Short	Change in Y position as $(currentY * 32 - prevY * 32) * 128$
			Delta Z	Short	Change in Z position as $(currentZ * 32 - prevZ * 32) * 128$
			On Ground	Boolean	

Float	4 bytes	Decimal number
Short	2 bytes	-32758 to 32767



Geometry Compression



```
void OutputMemoryBitStream::Write(const Quaternion& inQuat)
{
    float precision = (2.f / 65535.f);
    Write(ConvertToFixed(inQuat.mX, -1.f, precision), 16);
    Write(ConvertToFixed(inQuat.mY, -1.f, precision), 16);
    Write(ConvertToFixed(inQuat.mZ, -1.f, precision), 16);
    Write(inQuat.mW < 0);
}
```

```
void InputMemoryBitStream::Read(Quaternion& outQuat)
{
    float precision = (2.f / 65535.f);

    uint32_t f = 0;

    Read(f, 16);
    outQuat.mX = ConvertFromFixed(f, -1.f, precision);
    Read(f, 16);
    outQuat.mY = ConvertFromFixed(f, -1.f, precision);
    Read(f, 16);
    outQuat.mZ = ConvertFromFixed(f, -1.f, precision);

    outQuat.mW = sqrtf(1.f -
                    outQuat.mX * outQuat.mX +
                    outQuat.mY * outQuat.mY +
                    outQuat.mZ * outQuat.mZ );

    bool isNegative;
    Read(isNegative);

    if(isNegative)
    {
        outQuat.mW *= -1;
    }
}
```

Here's the code to turn a quaternion into the equivalent in three values.

Also, you might see that the code does fixed point compression (*ConvertToFixed* | *ConvertFromFixed*) because rotations do not typically need a lot of precision

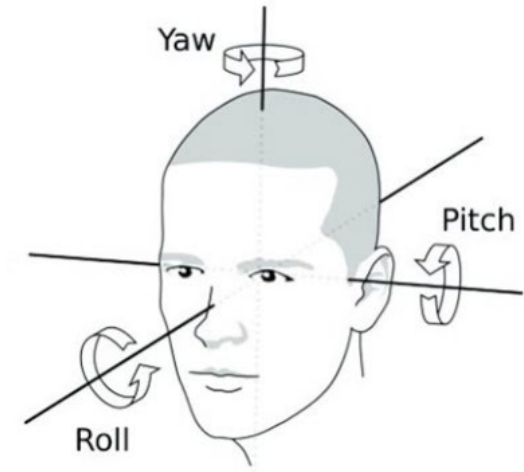
Geometry compression

Avoid unnecessary values

Set Player Rotation

Packet ID	State	Bound To	Field Name	Field Type	Notes
0x19	Play	Server	Yaw	Float	Absolute rotation on the X Axis, in degrees.
			Pitch	Float	Absolute rotation on the Y Axis, in degrees.
			On Ground	Boolean	True if the client is on the ground, false otherwise.

Also, no roll



Videos – Snapshot Compression



CITM

- https://youtu.be/cPvwUk1_uPU - Uncompressed
- <https://youtu.be/1sLUi0cXUqE> - Compressed orientation with fixed point precision
- https://youtu.be/evuQS8ZAU_c - Fixed-point velocity and at rest flag
- https://youtu.be/pY2xkWkJ_5U - Fixed-point position

Exercise – Homework

Do you remember the data structure for our avatar in a hypothetical FPS? It used 320 bits to transmit a single movement. Try to apply entropy compression, fixed point compression, and geometry compression. Make as many assumptions as you need

```
struct playerState {  
    vec3f position;  
    vec3f linear_velocity;  
    quat4f orientation;  
};
```





UNIVERSITAT POLITÈCNICA DE CATALUNYA
BARCELONATECH

Centre de la Imatge i la Tecnologia Multimèdia

Impact on programming

3.2 Surviving a harsh environment

Sergi Abadal (abadal@ac.upc.edu)

Xarxes i Jocs On-line – 2025-2026
Grau en Disseny i Desenvolupament de Videojocs



UNIVERSITAT POLITÈCNICA DE CATALUNYA
BARCELONATECH

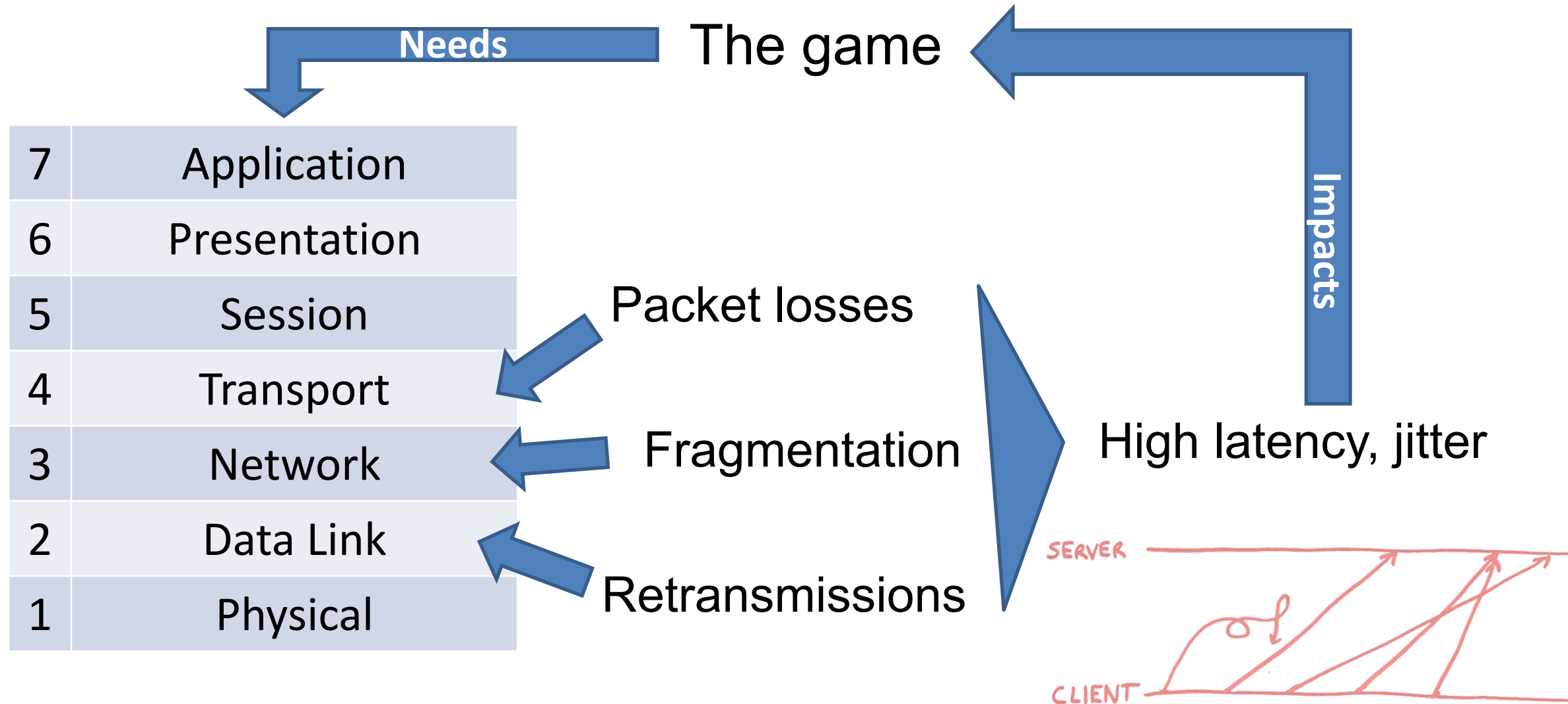
Centre de la Imatge i la Tecnologia Multimèdia

Extra material: Problems caused by the network (extended)

What causes problems?



CITM



- Physical and link layers
 - With 5G and beyond, cell phones keep improving their delays and bandwidth. Yet you generally cannot beat wired (also more stable).
 - Then it's clear that you cannot generate very high demands on mobile games.
 - You need to adapt and make tricks to go beyond the possibilities of the network in those cases.

7	Application
6	Presentation
5	Session
4	Transport
3	Network
2	Data Link
1	Physical

- Network layer
 - NAT problems and firewalls can prevent from having a fluid experience.
 - Segments from upper layers are treated independently, losing order.
 - Fragmentation can be detrimental to performance, generating extra packets.
 - Yet you have to live with them

7	Application
6	Presentation
5	Session
4	Transport
3	Network
2	Data Link
1	Physical

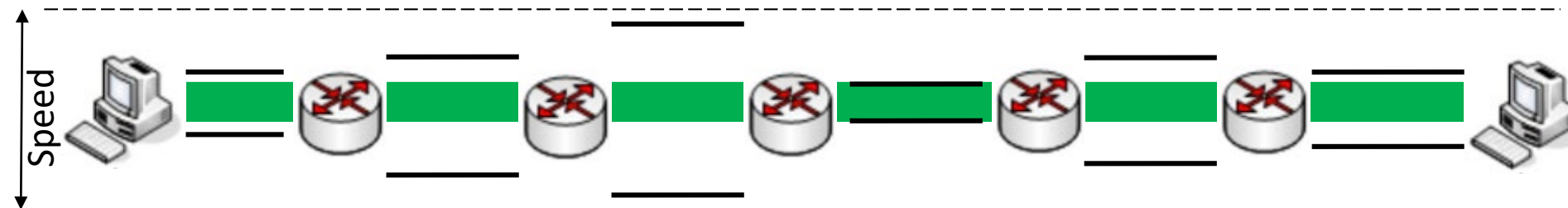
- **Example: fragmentation and MTU**
 - Programmers may want to try to avoid fragmentation and typical MTU is 1500B.
 - Data structures to send should be $< \text{MTU}$, but not too long
 - 100B data + 20B header $\rightarrow$ payload is 83.3%
 - 1480B data + 20B header $\rightarrow$ payload is 98.6%

7	Application
6	Presentation
5	Session
4	Transport
3	Network
2	Data Link
1	Physical

Impact of different layers

- Transport layer
 - There is a tradeoff between speed and reliability when choosing TCP or UDP. You may want critical data to run through TCP, and some less critical data to go with UDP.
 - Remember that you go as slow as the bottleneck of the whole path from source to destination

7	Application
6	Presentation
5	Session
4	Transport
3	Network
2	Data Link
1	Physical



- **Example: latency vs throughput in TCP**
 - Imagine that you need to send critical updates reliably and relatively fast. TCP seems appropriate and you cannot use UDP if you need reliability.
 - The **Neigh Algorithm** in TCP decides to group together small pieces of data within a single segment to improve the utilization.
 - If such updates are not frequent, you may not want to wait for this algorithm to fill a segment as you are making critical updates wait too much.

7	Application
6	Presentation
5	Session
4	Transport
3	Network
2	Data Link
1	Physical

- **Example: latency vs reliability in TCP**

- If you need to check ordering of packets or prevent errors, TCP seems appropriate.
- However, the ordering mechanism buffers segments until the complete segment is received. If a single packet is lost, then the whole segment is buffered until the missing packet is resent.
- The whole segment is given to the game at once, who needs to react to this burst of updates.

7	Application
6	Presentation
5	Session
4	Transport
3	Network
2	Data Link
1	Physical

- Presentation and session
 - Be aware that everyone can sniff packets. You lose control once the packet leaves your computer.
 - There has to be some security in the game. Make sure that there is no confusion about different game sessions, or avoid impersonation.
 - Example: micro-transaction data should be encrypted, while state updates not need to.
 - Data should be encoded efficiently (reducing bandwidth requirements) and in a way so that all parties can understand it

7	Application
6	Presentation
5	Session
4	Transport
3	Network
2	Data Link
1	Physical